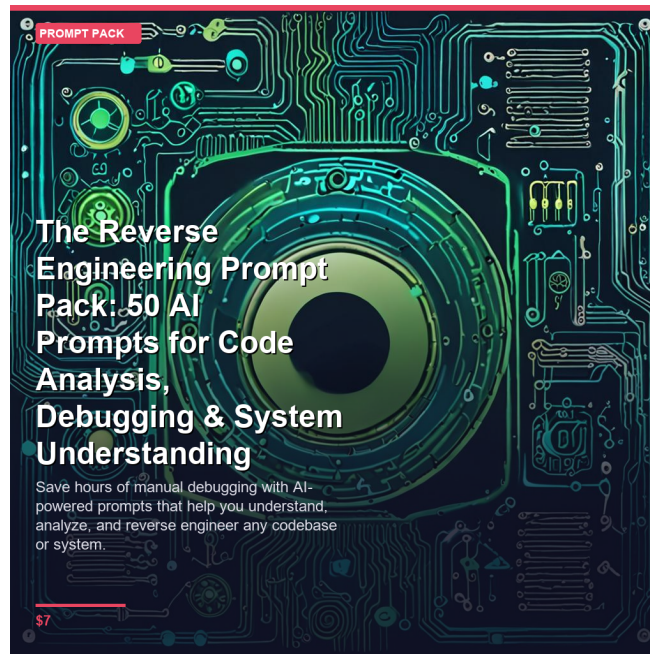




The Reverse Engineering Prompt Pack: 50 AI Prompts for Code Analysis, Debugging & System Understanding

Save hours of manual debugging with AI-powered prompts that help you understand, analyze, and reverse engineer any codebase or system.

For: Software engineers, security researchers, technical consultants, and developers working with legacy code or unfamiliar systems

What's Inside

- Section 1** — Introduction
- Section 2** — Code Comprehension & Architecture Analysis
- Section 3** — Debugging & Error Analysis
- Section 4** — Security & Vulnerability Assessment
- Section 5** — Legacy Code Understanding
- Section 6** — System Behavior & Testing
- Section 7** — Reverse Engineering & Analysis
- Section 8** — Optimization & Improvement
- Section 9** — Migration & Modernization
- Section 10** — Security Analysis Prompts
- Section 11** — Documentation Generation Prompts
- Section 12** — Integration & Compatibility Prompts
- Section 13** — Code Quality & Metrics Prompts
- Section 14** — Specialized Analysis Prompts
- Section 15** — Conclusion

SECTION 1

Introduction

This prompt pack contains 75 specialized prompts designed to help you quickly understand, analyze, and reverse engineer any codebase or system. Whether you're debugging legacy code, analyzing unfamiliar architectures, or conducting security research, these prompts will dramatically reduce the time you spend manually tracing through code.

How to use this pack: Copy any prompt and paste it into your AI assistant (ChatGPT, Claude, etc.), replacing bracketed placeholders with your specific code, error messages, or system details. For best results, provide context about your programming language, framework, and what you're trying to

achieve. Use these prompts iteratively—start broad with architecture analysis, then drill down into specific functions or bugs. Combine prompts for complex investigations.

SECTION 2

Code Comprehension & Architecture Analysis

■ Prompt 1: Quick Codebase Overview

"Analyze this codebase structure and provide a high-level overview: [paste directory structure or file list]. Identify the primary architecture pattern, main components, likely tech stack, and entry points. What does this system appear to do?"

■ Prompt 2: Architecture Pattern Detection

"Examine this code and identify the architectural pattern being used (MVC, microservices, layered, etc.): [paste code samples]. Explain how the pattern is implemented and any deviations from standard practice."

■ Prompt 3: Data Flow Mapping

"Trace the data flow from input to output in this code: [paste code]. Create a step-by-step explanation of how data transforms, which functions process it, and where it's stored or transmitted."

■ Prompt 4: Dependency Graph Explanation

"Analyze these import statements and dependencies: [paste imports/requirements]. Explain what each major dependency does, why it's likely needed, and identify any deprecated, redundant, or security-risk libraries."

■ Prompt 5: Entry Point Identification

"Given this codebase structure: [paste structure], identify all possible entry points (main functions, API endpoints, event handlers). Explain what triggers each entry point and what it initiates."

■ Prompt 6: Module Responsibility Breakdown

"Analyze this module/class: [paste code]. Using single responsibility principle, explain what this component's primary purpose is, what responsibilities it has, and whether it's doing too much."

■ Prompt 7: Hidden Functionality Discovery

"Review this code for non-obvious functionality: [paste code]. Identify any hidden features, debug code, commented-out logic, feature flags, or conditional behaviors that aren't immediately apparent."

■ Prompt 8: Configuration Analysis

"Analyze this configuration file: [paste config]. Explain each setting's purpose, identify critical parameters, security-relevant options, and any unusual or dangerous configurations."

■ Prompt 9: API Contract Extraction

"Extract the complete API contract from this code: [paste API code]. List all endpoints, methods, expected parameters, response formats, authentication requirements, and error codes."

■ Prompt 10: Database Schema Inference

"Based on this code: [paste data access code], reverse engineer the database schema. Describe tables, relationships, indexes, constraints, and any potential schema issues you notice."

SECTION 3

Debugging & Error Analysis

■ Prompt 11: Error Message Decoder

"Explain this error message in plain terms: [paste error]. What's the root cause, what common scenarios trigger it, and provide three specific troubleshooting steps prioritized by likelihood."

■ Prompt 12: Stack Trace Analysis

"Analyze this stack trace: [paste trace]. Identify the actual failure point, trace the call chain backwards, explain what each frame was attempting, and pinpoint where investigation should focus."

■ Prompt 13: Logic Bug Detective

"This code produces incorrect output: [paste code and expected vs actual behavior]. Identify logical errors, off-by-one issues, incorrect conditionals, or flawed assumptions causing the problem."

■ Prompt 14: Race Condition Identifier

"Review this concurrent code for race conditions: [paste code]. Identify shared resources, unsynchronized access points, potential timing issues, and explain scenarios where failures could occur."

■ Prompt 15: Memory Leak Hunter

"Analyze this code for memory leak vulnerabilities: [paste code]. Identify objects that might not be released, circular references, event listeners without cleanup, or resource allocation without deallocation."

■ Prompt 16: Performance Bottleneck Spotter

"Examine this code for performance issues: [paste code]. Identify inefficient algorithms, unnecessary loops, redundant operations, blocking calls, or resource-intensive operations that could be optimized."

■ Prompt 17: Edge Case Generator

"Given this function: [paste function], generate a comprehensive list of edge cases that could break it. Include boundary values, null/empty inputs, type mismatches, and unusual but valid scenarios."

■ Prompt 18: Null Reference Tracker

"Trace potential null/undefined reference errors in this code: [paste code]. Identify every place where null values could propagate, which variables lack null checks, and where NPEs could occur."

■ Prompt 19: Exception Flow Analysis

"Map the exception handling in this code: [paste code]. Show what exceptions can be thrown, where they're caught (or not), whether handling is appropriate, and identify unhandled failure modes."

■ Prompt 20: Intermittent Bug Hypothesizer

"This bug occurs randomly: [describe symptoms and paste code]. Generate hypotheses for intermittent failures including timing dependencies, environmental factors, state corruption, and testing strategies for each."

SECTION 4

Security & Vulnerability Assessment

■ Prompt 21: Security Vulnerability Scanner

"Analyze this code for security vulnerabilities: [paste code]. Identify injection flaws, authentication issues, insecure data handling, hardcoded secrets, and rate limiting problems. Prioritize by severity."

■ Prompt 22: Input Validation Audit

"Review this input handling code: [paste code]. Identify all user inputs, assess validation completeness, find bypasses or insufficient sanitization, and recommend specific validation improvements."

■ Prompt 23: Authentication Flow Analysis

"Examine this authentication implementation: [paste code]. Evaluate security strength, identify bypasses, check session management, assess password handling, and find authorization logic flaws."

■ Prompt 24: SQL Injection Detector

"Check this database code for SQL injection vulnerabilities: [paste code]. Identify all query construction points, assess parameterization, find string concatenation risks, and show secure alternatives."

■ Prompt 25: Cryptography Implementation Review

"Analyze this cryptographic code: [paste code]. Evaluate algorithm choices, key management, initialization vectors, randomness sources, and identify any implementation weaknesses or misuse of cryptographic primitives."

■ Prompt 26: Access Control Checker

"Review this authorization code: [paste code]. Map the permission model, identify privilege escalation risks, find missing authorization checks, and assess whether least-privilege principle is followed."

■ Prompt 27: API Security Assessment

"Evaluate this API implementation's security: [paste API code]. Check authentication, rate limiting, input validation, error message information disclosure, CORS configuration, and injection vulnerability protection."

■ Prompt 28: Sensitive Data Exposure Finder

"Scan this code for sensitive data exposure: [paste code]. Identify logging of secrets, cleartext storage, insecure transmission, insufficient encryption, and data leaks through error messages or responses."

■ Prompt 29: CSRF Vulnerability Check

"Analyze these endpoints for CSRF vulnerabilities: [paste endpoint code]. Identify state-changing operations, assess CSRF token implementation, check SameSite cookie attributes, and find unprotected endpoints."

■ Prompt 30: Deserialization Risk Analyzer

"Review this deserialization code: [paste code]. Identify unsafe deserialization of untrusted data, object injection risks, type confusion vulnerabilities, and recommend safer alternatives."

SECTION 5

Legacy Code Understanding

■ Prompt 31: Legacy Code Translator

"Explain this legacy code in modern terms: [paste old code]. Describe what it does, why it was written this way, what modern equivalents exist, and outline a refactoring approach."

■ Prompt 32: Obsolete Pattern Identifier

"Analyze this code for outdated patterns: [paste code]. Identify deprecated APIs, obsolete language features, old-style idioms, and suggest current best-practice replacements with justification."

■ Prompt 33: Undocumented Function Explainer

"This function has no documentation: [paste function]. Explain its purpose, parameters, return values, side effects, and what use case it serves within the broader system."

■ Prompt 34: Dead Code Detector

"Identify unused or dead code in this module: [paste code]. Find unreachable branches, unused functions, redundant conditionals, and code that could be safely removed."

■ Prompt 35: Magic Number Decoder

"This code contains magic numbers: [paste code]. Explain what each hard-coded number represents, why these values were chosen, and suggest named constants with appropriate descriptive names."

■ Prompt 36: Spaghetti Code Untangler

"Simplify this complex code logic: [paste tangled code]. Break down what it does, identify unnecessary complexity, and outline how to refactor it into cleaner, more maintainable components."

■ Prompt 37: Global State Mapper

"Map global state usage in this code: [paste code]. Identify all global variables, shared state, singletons, and explain dependencies. Assess risks and suggest encapsulation strategies."

■ Prompt 38: Comment-Code Mismatch Finder

"Compare comments to actual code behavior: [paste commented code]. Identify where comments are outdated, misleading, or incorrect. Suggest updated documentation reflecting actual functionality."

■ Prompt 39: Implicit Assumption Extractor

"Identify implicit assumptions in this code: [paste code]. What must be true for this to work correctly? What preconditions, environmental dependencies, or data invariants are assumed but not validated?"

■ Prompt 40: Refactoring Priority Assessor

"Evaluate this legacy module for refactoring: [paste code]. Score technical debt severity, identify highest-impact refactorings, estimate risk, and recommend which improvements to tackle first."

SECTION 6

System Behavior & Testing

■ Prompt 41: Test Case Generator

"Generate comprehensive test cases for this function: [paste function]. Include happy path, edge cases, error conditions, boundary values, and invalid inputs. Write in Given-When-Then format."

■ Prompt 42: Behavior Prediction

"Predict this code's behavior given these inputs: [paste code and input scenario]. Walk through execution step-by-step, explain state changes, and describe the final output or side effects."

■ Prompt 43: Regression Risk Analyzer

"Assess regression risk if this code changes: [paste code and proposed change]. Identify dependent components, potential breaking changes, and recommend testing scope to catch regressions."

■ Prompt 44: Side Effect Cataloger

"List all side effects of this function: [paste function]. Include database changes, file operations, network calls, global state modifications, logging, and any observable system changes."

■ Prompt 45: Integration Point Identifier

"Identify external integration points in this code: [paste code]. Map all API calls, database queries, file I/O, message queues, third-party services, and explain each integration's purpose."

■ Prompt 46: State Machine Extractor

"Extract the state machine from this code: [paste stateful code]. Define all states, transitions, triggering events, guards/conditions, and draw a textual state diagram."

■ Prompt 47: Concurrency Analysis

"Analyze concurrent execution of this code: [paste code]. Identify thread-safety issues, synchronization points, locks, potential deadlocks, and explain safe concurrent usage patterns."

■ Prompt 48: Failure Mode Analysis

"Enumerate failure modes for this system: [paste code or describe system]. For each component, identify how it can fail, cascading effects, and whether failures are handled gracefully."

■ Prompt 49: Timing Dependency Finder

"Identify timing-dependent behavior in this code: [paste code]. Find timeouts, delays, polling loops, race conditions, or any logic where execution timing affects correctness."

■ Prompt 50: Mocking Strategy Designer

"Design a mocking strategy for testing this code: [paste code]. Identify dependencies to mock, specify mock behaviors, explain what each mock should simulate, and justify the approach."

SECTION 7

Reverse Engineering & Analysis

■ Prompt 51: Algorithm Identifier

"Identify the algorithm implemented here: [paste code]. Name the algorithm, explain its time/space complexity, describe typical use cases, and assess whether this implementation is optimal."

■ Prompt 52: Protocol Reverse Engineer

"Reverse engineer the communication protocol from this code: [paste network/API code]. Document message formats, sequencing, handshakes, authentication, and create a protocol specification."

■ Prompt 53: Binary Logic Decoder

"Explain this bitwise operation code: [paste bit manipulation code]. Break down each operation, explain the bit-level logic, describe what data transformation occurs, and why this approach was used."

■ Prompt 54: Obfuscation Unraveler

"De-obfuscate this code: [paste obfuscated code]. Explain what it actually does, rename misleading variables meaningfully, simplify convoluted logic, and provide a cleaned-up equivalent."

■ Prompt 55: Design Pattern Detector

"Identify design patterns in this code: [paste code]. Name each pattern, explain its implementation, assess whether it's appropriately used, and identify any anti-patterns present."

■ Prompt 56: Third-Party Integration Analyzer

"Analyze how this code integrates with third-party service: [paste integration code]. Explain the API being used, authentication method, data exchanged, error handling, and identify integration risks."

■ Prompt 57: Configuration-Driven Behavior Mapper

"Map how configuration affects this code's behavior: [paste configurable code]. Identify all configuration-driven branches, explain behavioral differences, and document configuration dependencies."

■ Prompt 58: Encoding Scheme Identifier

"Identify the encoding/serialization scheme used here: [paste encoded data or encoding code]. Explain the format, how to decode it, why it was chosen, and show encoding/decoding examples."

■ Prompt 59: Business Logic Extractor

"Extract business rules from this code: [paste code]. Translate technical implementation into business logic rules, identify calculation formulas, validation rules, and workflow steps in domain language."

■ Prompt 60: Middleware Chain Analyzer

"Analyze this middleware/pipeline: [paste middleware code]. Explain execution order, what each component does, data transformations between stages, and short-circuit or branching conditions."

SECTION 8

Optimization & Improvement

■ Prompt 61: Code Smell Detector

"Identify code smells in this code: [paste code]. Point out long methods, duplicated code, large classes, feature envy, inappropriate intimacy, and suggest specific refactorings for each."

■ Prompt 62: Performance Optimization Suggester

"Suggest performance optimizations for this code: [paste code]. Identify algorithmic improvements, caching opportunities, lazy loading, database query optimization, and quantify expected impact for each."

■ Prompt 63: Memory Usage Optimizer

"Optimize memory usage in this code: [paste code]. Find memory-intensive operations, suggest data structure alternatives, identify unnecessary copies or allocations, and recommend streaming approaches where applicable."

■ Prompt 64: Database Query Optimizer

"Optimize this database query code: [paste query code]. Suggest index usage, query restructuring, N+1 query elimination, eager loading strategies, and explain performance impact of each change."

■ Prompt 65: Scalability Assessment

"Assess scalability of this code: [paste code]. Identify bottlenecks as load increases, suggest horizontal/vertical scaling strategies, find shared state issues, and recommend architecture changes for scale."

■ Prompt 66: Error Handling Improver

"Improve error handling in this code: [paste code]. Add missing try-catch blocks, suggest specific exception types, recommend logging improvements, and implement graceful degradation where appropriate."

■ Prompt 67: Code Readability Enhancer

"Improve readability of this code: [paste code]. Suggest better variable names, extract magic numbers to constants, break up complex expressions, add clarifying comments, and simplify control flow."

■ Prompt 68: API Design Critic

"Critique this API design: [paste API interface]. Assess naming consistency, RESTful compliance, parameter clarity, response structure, versioning strategy, and suggest improvements aligned with best practices."

■ Prompt 69: Testability Improver

"Improve testability of this code: [paste code]. Identify hard-to-test dependencies, suggest dependency injection points, recommend interface extraction, and show how to break dependencies for easier testing."

■ Prompt 70: Documentation Generator

"Generate comprehensive documentation for this code: [paste code]. Write clear function descriptions, parameter explanations, return value documentation, usage examples, and note any gotchas or prerequisites."

SECTION 9

Migration & Modernization

■ Prompt 71: Framework Migration Planner

"Plan migration from [old framework] to [new framework] for this code: [paste code]. Identify equivalent constructs, note breaking changes, suggest migration steps, and highlight compatibility concerns."

■ Prompt 72: Language Version Upgrader

"Modernize this code for [language version]: [paste old version code]. Use new language features, replace deprecated syntax, improve idioms, and explain benefits of each modernization."

■ Prompt 73: API Versioning Strategy

"Design an API versioning strategy for this endpoint: [paste API code]. Recommend versioning approach (URL/header), plan for backward compatibility, define deprecation process, and show implementation example."

■ Prompt 74: Dependency Upgrade Assessor

"Assess upgrading this dependency from [old version] to [new version]: [paste code using dependency]. Identify breaking changes, required code modifications, new features to adopt, and estimate migration effort."

■ Prompt 75: Cloud Migration Analyzer

"Analyze this code for cloud migration: [paste code]. Identify cloud-native patterns to adopt, stateful components needing refactoring, configuration externalization needs, and recommend cloud service mappings."

SECTION 10

Security Analysis Prompts

■ Prompt 76: Vulnerability Scanner

"Scan this code for security vulnerabilities: [paste code]. Identify SQL injection risks, XSS vulnerabilities, authentication flaws, authorization issues, and insecure dependencies. Rate severity for each finding and provide remediation code examples."

■ Prompt 77: Authentication Flow Auditor

"Audit this authentication implementation: [paste auth code]. Evaluate password handling, session management, token security, MFA implementation, and logout procedures. Identify weaknesses and recommend security improvements with code examples."

■ Prompt 78: Input Validation Checker

"Review input validation in this code: [paste code]. Identify unvalidated inputs, insufficient sanitization, type coercion issues, and boundary condition problems. Suggest comprehensive validation rules and show secure implementation patterns."

■ Prompt 79: Secrets Detection

"Scan for hardcoded secrets in this codebase: [paste code]. Identify API keys, passwords, tokens, connection strings, and encryption keys. Recommend secrets management solutions and show migration steps to secure storage."

■ Prompt 80: Authorization Logic Reviewer

"Analyze authorization logic: [paste code]. Check for privilege escalation risks, broken access control, insecure direct object references, and missing permission checks. Map authorization flow and suggest fixes."

SECTION 11

Documentation Generation Prompts

■ Prompt 81: API Documentation Generator

"Generate comprehensive API documentation for: [paste API code]. Include endpoint descriptions, request/response formats, authentication requirements, error codes, rate limits, and usage examples in OpenAPI format."

■ Prompt 82: Code Comment Generator

"Add inline documentation to this code: [paste code]. Generate clear comments explaining logic, parameters, return values, side effects, and edge cases. Follow language-specific documentation standards."

■ Prompt 83: Architecture Documentation Creator

"Create architecture documentation for this system: [paste code samples]. Describe components, their responsibilities, interactions, data flow, and design patterns. Generate C4 model diagrams (context, container, component)."

■ Prompt 84: README Generator

"Generate a comprehensive README for this project: [paste main code]. Include project description, installation instructions, usage examples, configuration options, API reference, contributing guidelines, and troubleshooting section."

■ Prompt 85: Changelog Generator

"Analyze code changes and generate changelog: [paste before/after code or git diff]. Categorize as features, fixes, breaking changes, or deprecations. Follow Keep a Changelog format."

SECTION 12

Integration & Compatibility Prompts

■ Prompt 86: Third-Party Integration Analyzer

"Analyze this third-party integration: [paste integration code]. Evaluate API usage patterns, error handling, rate limiting, retry logic, and data mapping. Identify risks and suggest improvements."

■ Prompt 87: Cross-Platform Compatibility Checker

"Check cross-platform compatibility: [paste code]. Identify platform-specific dependencies, file system operations, networking code, and UI elements. Suggest portable alternatives and conditional implementations."

■ Prompt 88: Browser Compatibility Analyzer

"Analyze browser compatibility: [paste JavaScript/CSS code]. Identify features requiring polyfills, vendor prefixes, or unsupported APIs. Recommend compatibility layer and specify minimum browser versions."

■ Prompt 89: Microservices Integration Mapper

"Map microservices integration points: [paste service code]. Identify service dependencies, communication patterns, data contracts, failure modes, and circuit breaker implementations. Suggest resilience improvements."

■ Prompt 90: Legacy System Interface Analyzer

"Analyze legacy system interface: [paste integration code]. Identify protocol limitations, data transformation needs, encoding issues, and error handling gaps. Design adapter pattern implementation."

SECTION 13

Code Quality & Metrics Prompts

■ Prompt 91: Code Complexity Analyzer

"Calculate complexity metrics for: [paste code]. Compute cyclomatic complexity, cognitive complexity, nesting depth, and function length. Identify overly complex sections requiring simplification with refactoring suggestions."

■ Prompt 92: Code Coverage Analyzer

"Analyze test coverage for: [paste code and tests]. Identify untested code paths, edge cases without tests, and low-coverage modules. Generate test cases for missing coverage."

■ Prompt 93: Code Duplication Detector

"Detect code duplication: [paste codebase sections]. Identify duplicate blocks, similar logic patterns, and copy-paste code. Suggest DRY refactoring using functions, classes, or modules."

■ Prompt 94: Maintainability Index Calculator

"Calculate maintainability index for: [paste code]. Assess based on complexity, code volume, documentation, and readability. Provide scored report and prioritized improvement recommendations."

■ Prompt 95: Technical Debt Assessor

"Assess technical debt in: [paste code]. Identify code smells, architectural issues, outdated patterns, missing tests, and documentation gaps. Estimate refactoring effort and prioritize by impact."

SECTION 14

Specialized Analysis Prompts

■ Prompt 96: Concurrency Issue Detector

"Analyze for concurrency issues: [paste multi-threaded code]. Identify race conditions, deadlock risks, improper synchronization, thread-safety violations, and resource contention. Suggest thread-safe implementations."

■ Prompt 97: Memory Management Analyzer

"Analyze memory management: [paste code]. Identify memory leaks, excessive allocations, missing cleanup, circular references, and inefficient memory usage. Recommend optimization strategies."

■ Prompt 98: Configuration Analysis

"Analyze configuration management: [paste config-related code]. Review configuration loading, validation, environment handling, secrets management, and default values. Suggest twelve-factor app compliance improvements."

■ Prompt 99: Logging & Observability Auditor

"Audit logging implementation: [paste code]. Evaluate log levels, structured logging, sensitive data exposure, log volume, and observability instrumentation. Recommend improvements for debugging and monitoring."

■ Prompt 100: Internationalization Checker

"Check internationalization readiness: [paste code]. Identify hardcoded strings, date/time formatting, number formatting, locale handling, and right-to-left support. Suggest i18n implementation strategy."

SECTION 15

Conclusion

This Reverse Engineering Prompt Pack provides 100 specialized prompts designed to accelerate code analysis, debugging, and system understanding across diverse scenarios. From initial codebase exploration to security auditing, performance optimization to modernization planning, these prompts serve as your AI-powered toolkit for tackling complex engineering challenges.

How to Use This Pack Effectively

Start with exploration prompts when encountering unfamiliar code. Use architecture visualization and dependency mapping to build mental models quickly.

Apply targeted analysis prompts based on your specific goals—whether debugging production issues, refactoring legacy systems, or conducting security reviews.

Combine multiple prompts for comprehensive analysis. For example, pair performance profiling with code quality metrics to prioritize optimization efforts.

Customize prompts by adjusting parameters, adding context, or requesting specific output formats that match your workflow and documentation standards.

Best Practices

- Provide complete, compilable code snippets for accurate analysis
- Include relevant context like framework versions and environment details
- Review AI suggestions critically and validate recommendations
- Iterate with follow-up questions to deepen understanding
- Document insights gained for team knowledge sharing

Final Note

Reverse engineering is both art and science. While AI accelerates analysis, your engineering judgment remains essential. Use these prompts as powerful accelerators that augment—not replace—your expertise, enabling you to understand complex systems faster and make better architectural decisions.

Thank you for your purchase.

Put this into action today — not tomorrow.

★ If this helped you, please leave a review. ★